

1. Pointer & Array Fundamentals

Snippet 1.1: String Modification via Pointers

```
char my_pet[] = "cat";
char *your_pet = my_pet;
your_pet[0] = 'r';
printf("%s %s\n", my_pet, your_pet);
```

A local character array `'my_pet'` is initialized with the string `"cat"`. A pointer `'your_pet'` is declared and assigned the value of `'my_pet'`. The character at index 0 of `'your_pet'` is altered to `'r'`, and both variables are printed.

In C, an array identifier naturally decays into a pointer to its first element when assigned to a pointer variable. Thus, both `'my_pet'` and `'your_pet'` reference the exact same memory address on the stack. Modifying the value via the pointer directly mutates the underlying stack memory buffer, transforming the string from `"cat"` to `"rat"` for both variables.

1.2: Pointer Arithmetic and Indirection Order

```
int array[] = {10, 20, 30, 40, 50};
int value = *array;
value++;
```

An integer array is allocated on the stack. The array identifier `'array'` acts as a pointer to the first index. The code dereferences it, assigns the value to a local standalone integer `'value'`, and increments `'value'`.

Dereferencing the array base symbol (`'*array'`) yields the value of the first element (10). Because this value is copied into a new variable `'value'`, executing `'value++'` only changes that isolated variable to 11 without altering the array elements. Shifting elements via `'*(array + n)'` scales the underlying memory address by `n * sizeof(int)` bytes.

1.3: Grave Mistake - Returning Local Stack Arrays

```
int* func() {
    int array[3] = {1, 2, 3};
    array[1] = 10;
    array[2] = 33;
    return array;
}
```

A function defines a local static-size array on its own stack frame, performs modifications, and attempts to return the address of that array back to the caller.

Stack memory frames are transient. As soon as a function returns, its stack frame is popped off and invalidated. The returned pointer becomes a dangling pointer, pointing to an unprotected block of memory that the operating system or subsequent function calls will immediately overwrite, causing unpredictable runtime crashes or memory corruption.

2. Type Conversions (Casting)

2.1: Implicit vs. Explicit Type Casting

```
int x = 4;
float y_implicit = x;
float y_explicit = (float)x;
```

An integer `'x'` is converted into a floating-point representation through an implicit assignment.

gnment conversion and an explicit unary cast operator conversion.

C permits scalar promotions. Implicit casting is managed directly by the compiler when upgrading a narrower variable type into a broader type. Explicit casting via the (type) operator enforces clarity and forces exact evaluations, which is highly practical in calculations like integer division to prevent unintended floor truncations.

2.2: Memory Truncation and Value Overflows

```
int x = 257;
char c = (char)x;

float pi = 3.14159f;
int pi_int = (int)pi;
```

A large integer 257 is cast down into a single-byte 'char'. Separately, a floating-point number is cast down into an integer type.

A standard character data type only spans 1 byte (0 to 255 for unsigned). Casting 257 (binary 100000001) causes an intentional integer overflow condition, stripping away the upper bits and wrapping the value around to 1. Casting a float to an int strips out all decimal point data entirely, truncating 3.14159 into an integer value of exactly 3.

2.3: Incompatible String-to-Int Raw Cast

```
char str[] = "123";
int num = (int)str;
```

A character string containing number digits is explicitly forced into a standard scalar integer type variable.

Direct pointer-to-integer casting does not interpret or parse ASCII text digits. Instead, it captures the raw, base 64-bit hexadecimal memory address pointing to the start of the string and saves that memory index number as an integer. To parse numbers out of text, standard library routines like 'atoi()' or 'strtol()' must be used instead.

2.4: Generic Void Pointers (void*) in Functions

```
void printValue(void *ptr, int type) {
    if (type == 1) {
        printf("%d\n", *(int*)ptr);
    }
}
```

A function takes a generic 'void*' argument, checks an explicit state indicator, and casts the generic address into a specific 'int*' layout type right before dereferencing it.

A 'void*' pointer is an abstract raw pointer that can hold any valid memory address. However, the compiler cannot naturally dereference a 'void*' because it lacks a clear byte size definition. Explicitly casting it to a targeted type (like 'int*') provides the compiler with the structural layout needed to safely read the bytes.

3. Dynamic Memory Management (The Heap)

3.1: Stack Allocation vs. Read-Only Data Segment

```
char str_array[] = "hello";
char *str_pointer = "hello";
```

A string is constructed locally using two distinct memory semantics: an inline array and

a direct pointer variable assignment pointing to a literal constant.

'str_array' dynamically copies the string literal text straight onto the current function stack context, giving the program read-write clearance over those specific stack bytes. Conversely, 'str_pointer' points directly to the original immutable string literal located inside the application's read-only text/data segment. Writing to 'str_pointer[0]' results in immediate memory access violations.

3.2: Dynamic Allocation and Mandatory NULL Check

```
char* createArray() {
    char *arr = malloc(5);
    if (arr == NULL) {
        return NULL;
    }
    return arr;
}
```

A block of 5 uninitialized bytes is explicitly allocated on the Heap. The pointer is checked to prevent operations on a NULL result, and then the address is returned safely to the outer environment.

Heap-allocated blocks are unbound from the local execution stack frame life cycle, remaining active until explicitly freed. If the system memory pool becomes completely exhausted, 'malloc()' fails and yields a 'NULL' pointer. Performing an explicit 'if (arr == NULL)' check is absolutely vital; skipping it and dereferencing a 'NULL' reference causes catastrophic program termination.

3.3: Allocation Size Mismatch Fallstruck

```
void* create_array(int size) {
    void *r = malloc(size * sizeof(int*));
    return r;
}
```

The function attempts to allocate an array intended to hold integer elements, but evaluates sizes utilizing the size of an integer pointer instead of a basic integer.

On standard modern architectures, a raw pointer size ('int*') spans 8 bytes, while a basic primitive integer ('int') spans 4 bytes. Passing a 'size = 3' will allocate $3 * 8 = 24$ bytes instead of the expected 12 bytes. While this provides enough raw space to avoid immediate out-of-bounds errors, it introduces silent inflation, wastes memory tracking pools, and breaks type consistency.

3.4: Cleared Allocation via calloc()

```
int *arr = calloc(5, sizeof(int));
```

Memory for a contiguous array block capable of holding 5 separate integer values is allocated on the heap.

Unlike 'malloc()', 'calloc()' takes two clean input parameters (element volume count and single unit byte size). Crucially, 'calloc()' executes a complete block-level zero-initialization sweep over the allocated memory range, ensuring the code does not read garbage data left behind by prior allocations.

3.5: Deallocation and the Dangling Pointer Hazard

```
int *arr = malloc(5 * sizeof(int));
// ... usage of the dynamic array ...
free(arr);
```

Dynamic memory is successfully requested on the heap and, upon finishing operations, is deallocated and recycled back to the kernel via `free()`.

Explicitly invoking `free()` ensures memory leaks are completely prevented, which can be verified using runtime analysis utilities like Valgrind. However, calling `free(arr)` does not modify or clear the inner address number value stored in the `arr` variable itself.

The pointer remains intact as a dangerous dangling pointer. Attempting to use or dereference `arr` after freeing it leads to severe undefined behaviors.